

PuenteLab - MANUAL_TECNICO

Manual Técnico — PuenteLab

1. Stack y versiones (fijas, sin + ni latest)

Componente	Versión
Kotlin	1.9.24
Android Gradle Plugin	8.4.2
Gradle	8.7 (wrapper)
KSP	1.9.24-1.0.20
Compose BOM	2024.06.00
Compose compiler extension	1.5.14
Room	2.6.1
Navigation Compose	2.7.7
Lifecycle	2.8.1
JDK	17
minSdk / targetSdk / compileSdk	24 / 34 / 34

Esta combinación es una de las combinaciones conocidas y estables de mediados de 2024 (Kotlin 1.9.x + Compose compiler 1.5.14 + AGP 8.4.x + Room 2.6.1 son mutuamente compatibles). **No se pudo verificar por compilación real en este entorno** (ver BUILD_REPORT.md); se recomienda que la primera ejecución de CI en GitHub confirme la resolución de dependencias.

2. Arquitectura

MVVM + Repository sobre tres paquetes:

```
data/
  local/          Room: entity/, dao/, converters/,
PuenteLabDatabase.kt
  repository/     ProfileRepository, CatalogRepository,
DesignRepository, SimulationRepository, Mappers.kt
  seed/           SeedMaterials, SeedVehicles, SeedChallenges,
SeedBadgesAndStamps, AvatarCatalog, DatabaseSeeder
domain/
  model/          Clases puras de Kotlin (sin Android/Room):
GridPoint, BridgeNode, BridgeMember,
                  BridgeChallengeSpec, BridgeDesignSpec,
SimulationResult, ChallengeAttempt, etc.
  logic/          BridgeEngine, ProgressEngine, BadgeEngine –
funciones puras, deterministas, testeadas
ui/
  theme/          Color.kt, Type.kt, Theme.kt
  components/     Ilustraciones Canvas (ScenarioScenes,
PivotCharacter, BadgeIcon, CollectibleComponents),
                  BuilderCanvasView (lienzo interactivo),
ProgressComponents, GridToCanvasMapper
  navigation/     Destinations.kt, NavGraph.kt
  screens/        Una carpeta por módulo (academy, escenarios,
materials, builder, designs, progress,
onboarding, profile)
  viewmodel/      Un ViewModel por pantalla + ViewModelFactory
```

(inyección manual)
util/ AppContainer.kt (contenedor manual de
dependencias)

Regla seguida estrictamente: los Composables nunca ejecutan SQL ni contienen reglas de negocio; los ViewModels exponen StateFlow y delegan en repositorios; los repositorios traducen entre entidades Room y modelos de dominio (Mappers.kt) y orquestan Room + motor de dominio; BridgeEngine/ProgressEngine/BadgeEngine no importan nada de Android — se pueden compilar y probar como Kotlin/JVM puro (así se hizo, ver BUILD_REPORT.md).

No se usa Hilt/Dagger: con 4 repositorios y ~14 entidades, un contenedor manual (AppContainer) resulta más simple de auditar que introducir un framework de inyección de dependencias completo. Es una decisión de alcance, no una limitación técnica.

3. Flujo de datos: “Probar puente”

1. BuilderScreen llama a BuilderViewModel.runSimulation(vehicleId, weightMultiplier).
2. El ViewModel delega en SimulationRepository.runSimulation(design, challenge, vehicleId, multiplier).
3. El repositorio carga el mapa de materiales desde Room, ejecuta BridgeEngine.simulate(...) (motor de dominio puro), y persiste:
 - una fila en simulation_runs (el intento),
 - una fila en simulation_results por cada barra analizada,
 - actualiza progress (estado del desafío),
 - recalcula XP/nivel con ProgressEngine.totalXp(...) sobre **todo** el historial persistido,
 - recalcula insignias con BadgeEngine.unlockedBadges(...) y guarda las nuevas en user_badges,
 - desbloquea sellos de builder_stamps cuyo unlockChallengeId/unlockBadgeId corresponda.
4. El resultado (SimulationResult) vuelve al ViewModel, que actualiza el StateFlow y dispara SimulationResultDialog (animación de cruce + retroalimentación + estrellas).

Ninguna parte de este flujo usa aleatoriedad ni valores inventados: todo se deriva del diseño real del jugador y del historial persistido.

4. BridgeEngine — motor de simulación

Ver comentario KDoc en el propio archivo (domain/logic/BridgeEngine.kt) para el detalle algorítmico completo. Resumen:

1. **Conectividad** (BFS sobre grafo de todas las barras): orilla izquierda <-> orilla derecha.
2. **Ruta de calzada:** BFS solo sobre barras de rol DECK cuya pendiente $\leq \text{maxSlope}$ del nivel.
3. **Presupuesto:** costo = suma de(longitud x costo/unidad del material) + apoyos de pago (35 c/u).
4. **Capacidad de carga por barra:** capacidad = resistencia_material x bono / factor_longitud.
 - factor_longitud penaliza tramos libres > 3 unidades (+18 % por unidad extra).
 - Bonos geométricos reales (no declarados por el jugador, **detectados** por el motor):
 - Cercha: +55 % si el nodo toca una barra de rol BRACE.
 - Arco: +60 % a toda la ruta si algún nodo interior está claramente más alto que ambas orillas.
 - Suspensión: +80 % si el nodo toca una barra de rol CABLE.
 - Carga muerta: cada barra suma peso_material x longitud a su propia demanda.

5. **Veredicto y estrellas:** aprueba si no hay ninguna barra con demanda > capacidad y el costo no excede el presupuesto; estrellas según margen de presupuesto y esfuerzo máximo.

5. Pantallas y su ViewModel

Pantalla	ViewModel	Repositorios usados
Onboarding	— (estado local)	—
Perfil (alta)	ProfileViewModel	ProfileRepository
Academia (hub)	AcademyViewModel	CatalogRepository, ProfileRepository, UserBadgeDao
Escenarios	ScenariosViewModel	CatalogRepository, ProgressDao
Materiales	MaterialsViewModel	CatalogRepository
Constructor	BuilderViewModel	CatalogRepository, DesignRepository, SimulationRepository
Mis diseños	DesignsViewModel	DesignRepository, CatalogRepository
Progreso	ProgressViewModel	CatalogRepository, ProgressDao, UserBadgeDao, StampDao, ProfileRepository

6. Identidad visual sin recursos externos

Todas las ilustraciones son código Kotlin (Compose Canvas/Path), no imágenes descargadas ni empaquetadas como PNG salvo el ícono de lanzador (generado localmente con Pillow en `tools/generate_launcher_icons.py`, solo para las densidades pre-Android 8 que no soportan icono adaptativo vectorial):

- `ScenarioScenes.kt`: 5 escenas (río, cañón, bosque, ciudad, montaña).
- `PivotCharacter.kt`: mascota guía con 3 estados de ánimo.
- `BadgeIcon.kt` / `CollectibleComponents.kt`: 9 insignias + 12 sellos, cada uno con glifo propio.
- `BuilderCanvasView.kt`: lienzo interactivo del constructor (grid, nodos, barras coloreadas por rol).

7. Pruebas

59 tests en `app/src/test`:

- `BridgeEngineTest.kt` (21), `ProgressEngineTest.kt` (11), `BadgeEngineTest.kt` (17): lógica de dominio pura. **Verificados con ejecución real** compilando exactamente estos archivos (más los modelos de los que dependen) con `kotlinc 1.9.24` fuera de Gradle/Android, usando un arnés de aserciones equivalente (no JUnit, ya que no había forma de obtener el jar de JUnit sin acceso a Maven Central). Las 56 aserciones equivalentes pasaron. Detalle completo en `BUILD_REPORT.md`.
- `RoomPersistenceTest.kt` (10): Room en memoria vía Robolectric. Sigue la API real de Room 2.6.1 / Robolectric 4.13 pero **no se ejecutó** en este entorno (sin Android SDK). Debe correr con `./gradlew testDebugUnitTest` en un entorno con SDK de Android.

8. Mantenimiento y ampliación

- **Añadir un material:** agregar una fila a `SeedMaterials.all`; el motor lo usa automáticamente en cuanto un `BridgeMember` lo referencia.
- **Añadir desafíos:** extender `SeedChallenges` (agregar un `ScenarioFlavor` nuevo, o más niveles a 1..9); revisar el balance con el mismo método usado para anclar la fórmula actual (ver `BUILD_REPORT.md`, sección de verificación de balance).
- **Añadir una insignia:** agregar un caso a `BadgeId`, una entrada a `BadgeEngine.catalog` y su regla de desbloqueo en `BadgeEngine.unlockedBadges`; `SeedBadges` la recoge automáticamente.
- **Cambiar el esquema de Room:** subir `version` en `@Database`, escribir una `Migration` (no hay ninguna todavía porque es la versión 1) y añadirla en `PuenteLabDatabase`.

9. Permisos

Ninguno declarado. No hay `INTERNET`, `CAMERA`, `RECORD_AUDIO`, `ACCESS_FINE_LOCATION` ni similares en `AndroidManifest.xml`.